

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

П РА К Т И Ч Е С К А Я Р А Б О Т А № 4

Разработка кода через тестирование поведения
по дисциплине «Методы тестирования программного обеспечения»

Выполнил
студент гр. з5130903/40002

И.Р. Фёдорова

Руководитель
старший преподаватель ВШ ПИ

В.А. Пархоменко

СОДЕРЖАНИЕ

Введение	3
1 BDD-тестирование.....	4
1.1 Основы методологии BDD и язык Gherkin.....	4
1.2 Выбор BDD-фреймворка	5
1.3 Описание тестовых случаев и BDD-сценариев	6
1.3.1 Сводная таблица тестового покрытия.....	7
1.4 Выводы	7
2 Описание экспериментов	8
2.1 Условия проведения экспериментов	8
2.2 Алгоритмы генерации наборов данных	9
2.3 Предварительный эксперимент по определению границ производительности.....	10
2.4 Наборы данных	11
2.5 Профилирование.....	12
3 Анализ результатов экспериментов	13
3.1 Сводные таблицы времени выполнения	13
3.2 Графики зависимостей времени выполнения от размерности	13
3.3 Анализ выявленных закономерностей	14
3.4 Сравнительная таблица применимости алгоритмов	14
3.5 Выводы	15
Заключение	16
Список использованных источников.....	18
Приложение 1 Листинги кода классов предварительного эксперимента.....	19
Приложение 2 Листинги BDD-сценариев и реализации шагов	23

ВВЕДЕНИЕ

Поведенческая разработка через тестирование (Behaviour-Driven Development, BDD) является развитием TDD, в котором основной акцент переносится с проверки отдельных методов на описание поведения системы языком, понятным разработчику, тестировщику и пользователю. Вместо технических названий тестов BDD предлагает формулировать исполняемые спецификации в формате пользовательских историй и сценариев Given–When–Then.

Практическая работа посвящена применению BDD к консольному приложению на Node.js, реализующему наукоемкую задачу обработки строк: поиск палиндромных подстрок алгоритмом Манакера. В приложении присутствуют два способа решения задачи: основной алгоритм Манакера со сложностью $O(n)$ и эталонный алгоритм расширения от центра со сложностью $O(n^2)$ в худшем случае. Это позволяет не только проверять корректность результата, но и проводить нагрузочное сравнение двух алгоритмических подходов.

Цель работы — разработать BDD-сценарии для ключевых модулей приложения, связать их с исполняемыми шагами Cucumber.js и провести экспериментальное сравнение производительности алгоритмов на наборах данных разного типа и размера.

Для достижения цели решены следующие задачи:

- A. изучены причины перехода от TDD к BDD и базовый синтаксис языка Gherkin;
- B. выбран фреймворк Cucumber.js как BDD-инструмент для проекта на Node.js;
- C. разработаны три Gherkin-истории для анализа палиндромов, JSON-ввода–вывода и нагрузочного сравнения;
- D. реализованы шаги BDD-сценариев, связывающие текстовые спецификации с кодом приложения;
- E. подготовлены генераторы строковых датасетов и проведён предварительный эксперимент;
- F. выполнено профилирование времени выполнения алгоритма Манакера и алгоритма расширения от центра;
- G. сформулированы выводы о соответствии теоретической сложности наблюдаемым результатам и о сравнении BDD с модульными тестами.

1 BDD-ТЕСТИРОВАНИЕ

Данная глава посвящена планированию и реализации BDD-тестирования (Behaviour-Driven Development) приложения поиска палиндромных подстрок. Описывается методология BDD, синтаксис языка Gherkin, выбор инструментария, а также разработанные тестовые сценарии для трёх ключевых функциональных областей системы.

1.1 Основы методологии BDD и язык Gherkin

BDD (Behaviour-Driven Development, разработка через поведение) — методология тестирования программного обеспечения, расширяющая концепцию TDD (Test-Driven Development). Ключевое отличие BDD заключается в том, что тесты описывают поведение системы с точки зрения бизнес-требований и пользователей, а не детали внутренней реализации [1].

Сценарии BDD записываются на предметно-ориентированном языке Gherkin — структурированном псевдоестественном языке, понятном как разработчикам, так и нетехническим участникам проекта. Основу синтаксиса Gherkin составляют ключевые слова, представленные в табл.1.1.

Таблица 1.1

Основные ключевые слова языка Gherkin

Ключевое слово	Назначение
Feature	Описание тестируемой функциональности и пользовательской истории
Rule	Бизнес-правило, объединяющее группу сценариев
Background	Предусловия, общие для всех сценариев в блоке Rule или Feature
Scenario	Конкретный тестовый случай с уникальным названием
Scenario Outline	Параметризованный сценарий с набором примеров Examples
Given	Описание начального состояния системы, то есть предусловие
When	Описание действия, инициируемого актором
Then	Описание ожидаемого результата или постусловия
And/But	Продолжение предыдущего шага того же типа
Examples	Таблица входных данных для Scenario Outline

Каждый Feature-файл содержит пользовательскую историю в формате: “Как <актор>, я хочу <действие>, чтобы <выгода>”. Такой формат гарантирует, что тест

описывает реальную потребность пользователя, а не произвольную техническую проверку.

В Gherkin поддерживаются два стиля написания шагов. Декларативный стиль описывает, что происходит в системе, абстрагируясь от деталей интерфейса:

```
Given строка равна "abacaba"
When выполняется анализ алгоритмом Манакера
Then самый длинный палиндром равен "abacaba"
```

Императивный стиль перечисляет конкретные действия пользователя:

```
Given пользователь ввел строку "abbaabba"
When пользователь выбирает пункт "Запустить алгоритм Манакера"
And пользователь выбирает пункт "Показать подробный отчет"
5 Then отчет содержит самый длинный палиндром "abbaabba"
```

В проекте оба стиля применяются осознанно: декларативный — для проверки алгоритмического результата, императивный — для интеграционных сценариев, приближенных к действиям пользователя в консольном приложении.

1.2 Выбор BDD-фреймворка

Для реализации BDD-тестов в проекте на JavaScript был выбран фреймворк Cucumber.js [2] в связке со встроенным модулем assert. Обоснование выбора приведено в табл.1.2.

Таблица 1.2

Сравнение BDD-фреймворков для JavaScript

Критерий	Cucumber.js	jest-cucumber	CodeceptJS
Язык сценариев	Gherkin (.feature)	Gherkin (.feature)	Gherkin / текстовые сценарии
Интеграция с Node.js	Нативная	Через Jest	Через собственный раннер
Параметризация	Scenario Outline + Examples	Аналогично	Аналогично
Отчёты	Pretty, HTML, JSON	Отчёты Jest	Встроенные HTML–отчёты
Сложность настройки	Низкая	Средняя	Средняя
Распространённость	Высокая	Средняя	Средняя

Cucumber.js был предпочтён благодаря прямой поддержке Gherkin, минимальной конфигурации и возможности хранить сценарии в отдельных

.feature-файлах. Зависимость подключена через npm: @cucumber/cucumber. Запуск тестов осуществляется командой `npm run bdd`. Контекст сценария инкапсулирован в классе `PalindromeWorld`, а реализации шагов размещены в `features/step_definitions/palindromeSteps.js`.

1.3 Описание тестовых случаев и BDD-сценариев

Разработанные BDD-тесты охватывают три функциональные области, каждая из которых реализована в отдельном .feature-файле.

Feature: Анализ палиндромных подстрок (`palindrome_analysis.feature`). Данный feature-файл тестирует вычислительную часть приложения: алгоритм Манакера, восстановление самого длинного палиндрома и подсчёт количества палиндромных подстрок. Используется декларативный стиль и конструкция `Rule + Background`. Актором является пользователь консольного приложения, который хочет получить понятный отчёт о симметричности строки.

Файл содержит два типа сценариев. Первый тип — `Scenario Outline` с таблицей `Examples`; он проверяет строки `abacaba`, `abba`, `abc` и `aaaaa`. Для каждой строки проверяются самый длинный палиндром, количество палиндромных подстрок и совпадение с эталонным алгоритмом расширения от центра. Второй тип — императивный сценарий, в котором пользователь вводит строку, запускает алгоритм Манакера и открывает подробный отчёт. Сценарии основного правила приведены в табл. 1.3.

Таблица 1.3

Сценарии анализа палиндромных подстрок

Сценарий	Given / When	Then
Декларативная проверка	Строка анализируется алгоритмом Манакера	Самый длинный палиндром и число палиндромов совпадают с ожидаемыми значениями
Сравнение с эталоном	Для той же строки запускается расширение от центра	Оба алгоритма возвращают одинаковый самый длинный палиндром
Императивный сценарий меню	Пользователь вводит строку и выбирает пункты меню	Отчёт содержит исходную строку и найденный палиндром, ошибки нет

Feature: Загрузка исходной строки и сохранение результата (`json_io.feature`). Данный feature-файл объединён бизнес-идеей надёжно-

го обмена данными через JSON-файлы. Актором является пользователь, которому нужно повторять эксперименты и передавать результаты без ручного копирования. Файл демонстрирует смешанный стиль написания шагов.

Первый сценарий проверяет корректную загрузку JSON-файла с полем `input`, запуск алгоритма Манакера и сохранение результата в файл `result.json`. Второй сценарий проверяет ошибку формата: если JSON не содержит обязательного поля `input`, приложение выдаёт понятную ошибку и не завершается аварийно.

Feature: Сравнение производительности алгоритмов (`performance_comparison`)
 Данный feature-файл тестирует сравнение алгоритма Манакера и алгоритма расширения от центра по корректности и времени выполнения. Актором является исследователь наукоёмкой задачи обработки строк. Сценарий реализован как `Scenario Outline` с тремя группами данных: `alternate`, `mirror` и `uniform`. Для каждой комбинации типа строки и длины проверяется совпадение результата двух алгоритмов, а время Манакера ограничивается пороговым значением. Методика выбора наборов данных и лимитов рассмотрена в главе 2.

1.3.1 Сводная таблица тестового покрытия

В табл.1.4 приведена сводная информация по всем разработанным BDD-сценариям.

Таблица 1.4

Сводное покрытие BDD-сценариев

Feature-файл	Сценариев	Наборов	Тестируемый модуль
<code>palindrome_analysis.feature</code>	5	4	<code>services</code> , <code>analyzers</code>
<code>json_io.feature</code>	2	2	<code>io</code>
<code>performance_comparison.feature</code>	3	3	<code>scripts</code> , <code>analyzers</code>
Итого	10	9	—

1.4 Выводы

В данной главе разработан план BDD-тестирования приложения поиска палиндромных подстрок. Рассмотрены основы языка Gherkin: ключевые слова, конструкция `Rule + Background`, параметризация через `Scenario Outline` и два стиля написания шагов — декларативный и императивный. В качестве фреймворка выбран `Cucumber.js`, обоснованный нативной интеграцией с `Node.js` и низким по-

рогом настройки. Разработаны 10 сценариев в трёх feature-файлах, охватывающих модули алгоритмов, JSON-ввода-вывода и нагрузочного сравнения.

2 ОПИСАНИЕ ЭКСПЕРИМЕНТОВ

Данная глава посвящена описанию экспериментов по измерению производительности алгоритмов поиска палиндромных подстрок — алгоритма Манакера и расширения от центра. Приводятся условия проведения экспериментов, методология генерации строковых наборов данных, описание датасетов, а также результаты профилирования.

2.1 Условия проведения экспериментов

Все эксперименты проводились в локальном проекте `palindrome_app`. Для измерения времени использовался встроенный модуль `Node.js perf_hooks`, а именно функция `performance.now()` [3]. Генерация строк выполнялась до запуска измеряемых алгоритмов, чтобы не смешивать время подготовки данных и время решения задачи.

Аппаратная конфигурация экспериментального стенда приведена в табл.2.1.

Таблица 2.1

Аппаратная конфигурация экспериментального стенда

Компонент	Характеристика
Процессор	Локальная рабочая станция пользователя
Оперативная память	Доступная оперативная память рабочей станции
Накопитель	Локальный SSD/HDD, каталог проекта <code>C:\mtpro-lab2</code>

Программная конфигурация приведена в табл.2.2.

Таблица 2.2

Программная конфигурация экспериментального стенда

Компонент	Версия / параметр
Операционная система	Windows
Среда выполнения	Node.js v24.13.0
npm	11.6.2
Unit-тесты	Jest

Компонент	Версия / параметр
BDD-фреймворк	Cucumber.js
Единицы измерения	миллисекунды

2.2 Алгоритмы генерации наборов данных

Для нагрузочного тестирования применялась программная генерация синтетических строк. Генерация реализована в модуле `scripts/stringDataset.js` и запускается скриптом `scripts/generateDatasets.js`. Всего используются четыре типа строк, каждый из которых моделирует отдельный класс входных данных.

Строка из одинаковых символов (uniform). Строка строится повторением символа `a`. Такая структура является наиболее тяжёлой для алгоритма расширения от центра: почти каждый центр можно расширять на большое расстояние, поэтому число сравнений стремится к квадратичному.

Чередующаяся строка (alternate). Строка строится повторением шаблона `ab`. Она содержит большое количество нечётных палиндромов и хорошо показывает рост нагрузки при регулярной структуре.

Зеркальная строка (mirror). Строка формируется из фрагмента и его обратной копии. Такой набор содержит крупный палиндромный блок и проверяет корректность восстановления длинного палиндрома.

Псевдослучайная строка (random-like). Строка строится повторением алфавитного шаблона. В ней мало длинных палиндромов, поэтому расширение от центра часто быстро останавливается. Этот тип используется как контрольный случай.

Сводные характеристики типов строк приведены в табл.2.3.

Таблица 2.3

Типы и параметры генерируемых строк

Тип строки	Идентификатор	Используемые длины
Одинаковые символы	uniform	4000; 16000
Чередование	alternate	1000; 5000
Зеркальная строка	mirror	2000; 10000
Псевдослучайная строка	random-like	10000

2.3 Предварительный эксперимент по определению границ производительности

До проведения основных BDD-тестов был выполнен предварительный эксперимент, цель которого — установить практические границы применимости каждого алгоритма и обосновать лимиты времени, заданные в нагрузочных сценариях. Исходный код скриптов, реализующих эксперимент, приведён в приложении 1.

Методика измерений. Для каждой комбинации типа строки и длины выполнялись следующие действия:

- А. генерация строки заданного типа;
- В. запуск алгоритма Манакера;
- С. запуск алгоритма расширения от центра;
- Д. проверка совпадения найденного самого длинного палиндрома;
- Е. фиксация времени выполнения и коэффициента ускорения.

Время генерации строки не включалось во время работы алгоритмов. Измерение проводилось в функции `measureAlgorithms`. Результаты предварительного эксперимента приведены в табл.2.4.

Таблица 2.4

Время выполнения алгоритмов в предварительном эксперименте

Набор	Тип	Длина	Манакер, мс	Центр, мс	Быстрее
ALT_1000_A	alternate	1000	0.547	1.066	Манакер
ALT_5000_A	alternate	5000	0.349	14.598	Манакер
MIR_2000_A	mirror	2000	0.420	0.989	Манакер
MIR_10000_A	mirror	10000	0.422	16.049	Манакер
UNI_4000_A	uniform	4000	0.221	17.121	Манакер
UNI_16000_A	uniform	16000	1.419	270.883	Манакер
RND_10000_A	random-like	10000	0.738	0.616	Центр

Итоговые границы производительности приведены в табл.2.5. Эти данные послужили основой для выбора размерностей и лимитов времени в нагрузочных BDD-сценариях `performance_comparison.feature`.

Таблица 2.5

Границы производительности алгоритмов

Тип строки	Граница	Причина
uniform	16000 символов	Худший случай для расширения от центра, разница уже превышает 190 раз

Тип строки	Граница	Причина
alternate	5000 символов	Регулярная структура, заметный рост времени эталона
mirror	10000 символов	Крупный палиндромный блок, результат обоих алгоритмов совпадает
random-like	10000 символов	Контрольный случай, расширение быстро останавливается

2.4 Наборы данных

В проекте применяется двухуровневая стратегия организации наборов данных, обусловленная различием целей функционального и нагрузочного тестирования.

Функциональные датасеты. Функциональные BDD-сценарии проверяют корректность чтения JSON-файла и сохранения результата. Для них данные создаются во временной директории непосредственно в шаге сценария, что позволяет тестировать полный путь: JSON-файл на диске — загрузка — анализ — сохранение результата.

Нагрузочные датасеты. Нагрузочные тесты проверяют производительность алгоритмов, поэтому строки генерируются детерминированно. Скрипт `generateDatasets.js` создаёт реальные JSON-файлы в каталоге `datasets`. Кодировка имён строится по шаблону `TYPE_SIZE_SERIES`. Расшифровка приведена в табл.2.6.

Таблица 2.6

Кодировка названий нагрузочных датасетов

Обозначение	Расшифровка
UNI	строка из одинаковых символов, тип <code>uniform</code>
ALT	строка с чередованием символов, тип <code>alternate</code>
MIR	зеркальная строка, тип <code>mirror</code>
RND	псевдослучайная строка, тип <code>random-like</code>
Число в середине	длина строки в символах
A	первая серия генерации

Список подготовленных файлов приведён в табл.2.7.

Таблица 2.7

Нагрузочные датасеты

Имя файла	Тип	Длина	Назначение
ALT_1000_A.json	alternate	1000	Малый регулярный набор
ALT_5000_A.json	alternate	5000	Средний регулярный набор
MIR_2000_A.json	mirror	2000	Зеркальная строка
MIR_10000_A.json	mirror	10000	Крупный зеркальный набор
UNI_4000_A.json	uniform	4000	Тяжёлый случай для эталона
UNI_16000_A.json	uniform	16000	Максимальный набор эксперимента
RND_10000_A.json	random-like	10000	Контрольный набор

2.5 Профилирование

Профилирование в рамках BDD-тестирования представляет собой сбор и анализ временных метрик непосредственно во время выполнения тестовых сценариев. В данном проекте профилирование реализовано на уровне скрипта `runPerformanceExperiment.js` и встроено в сценарий `performance_comparison.feature`.

Выполнение каждого нагрузочного сценария разбивается на три фазы:

- А. генерация строки заданного типа;
- В. выполнение алгоритма Манакера;
- С. выполнение алгоритма расширения от центра и сравнение результата.

После запуска команды `npx run perf` формируются файлы `reports/performance` и `reports/performance/performance-results.csv`. Профиль выполнения нагрузочных сценариев приведён в табл.2.8.

Таблица 2.8

Профиль выполнения нагрузочных сценариев

Набор	Тип	Длина	Манакер, мс	Центр, мс	Память, МБ
ALT_1000_A	alternate	1000	0.547	1.066	4.391
ALT_5000_A	alternate	5000	0.349	14.598	4.744
MIR_2000_A	mirror	2000	0.420	0.989	4.227
MIR_10000_A	mirror	10000	0.422	16.049	4.900
UNI_4000_A	uniform	4000	0.221	17.121	4.799
UNI_16000_A	uniform	16000	1.419	270.883	4.906
RND_10000_A	random-like	10000	0.738	0.616	6.007

Профилирование показывает, что на регулярных и особенно односимвольных строках доминирует время эталонного алгоритма расширения от центра. На псевдослучайной строке RND_10000_A оба алгоритма близки по времени, так как расширение почти сразу останавливается на большинстве центров.

3 АНАЛИЗ РЕЗУЛЬТАТОВ ЭКСПЕРИМЕНТОВ

В данной главе приведены результаты измерений производительности двух алгоритмов и выполнен анализ их соответствия теоретической сложности.

3.1 Сводные таблицы времени выполнения

Результаты запуска `node scripts/runPerformanceExperiment.js` приведены в табл.3.1. Все измерения выполнены на одних и тех же входных строках; поле `sameResult` во внутреннем отчёте равно `true` для всех наборов.

Таблица 3.1

Результаты сравнения алгоритмов

Датасет	Тип	Длина	Манакер, мс	Центр, мс	Ускорение
ALT_1000_A	alternate	1000	0.547	1.066	1.95
ALT_5000_A	alternate	5000	0.349	14.598	41.83
MIR_2000_A	mirror	2000	0.420	0.989	2.36
MIR_10000_A	mirror	10000	0.422	16.049	38.01
UNI_4000_A	uniform	4000	0.221	17.121	77.58
UNI_16000_A	uniform	16000	1.419	270.883	190.83
RND_10000_A	random-like	10000	0.738	0.616	0.84

3.2 Графики зависимостей времени выполнения от размерности

На рис.3.1 приведено визуальное сравнение времени выполнения. Значения построены по данным табл.3.1; для читаемости график показывает наиболее показательные наборы.

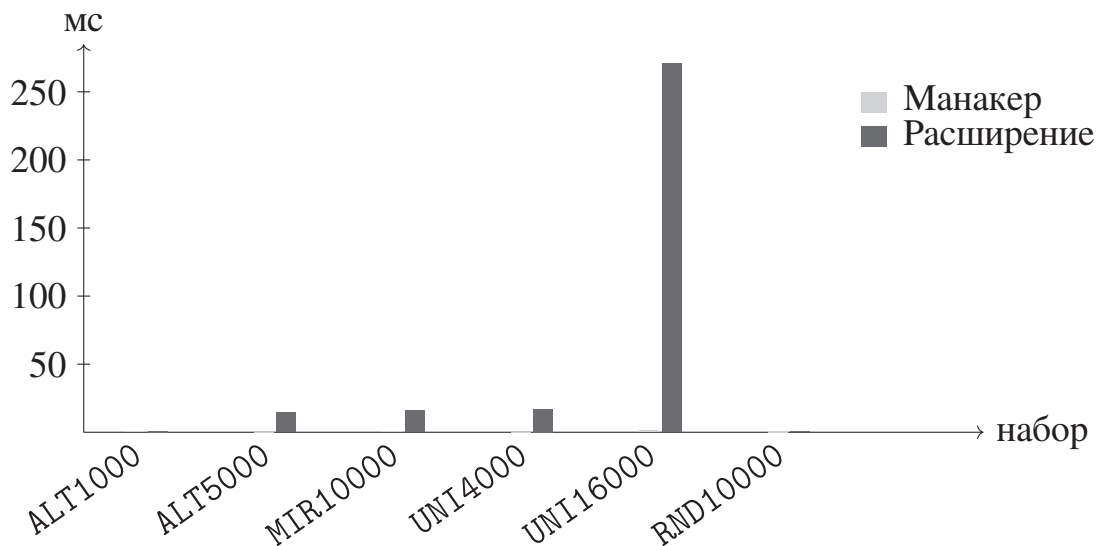


Рис.3.1. Сравнение времени выполнения алгоритмов

3.3 Анализ выявленных закономерностей

Полученные данные согласуются с теоретическими оценками. Алгоритм Манакера на всех наборах выполняется за время порядка единиц миллисекунд, так как каждый символ участвует в ограниченном числе операций. Эталонный алгоритм расширения от центра чувствителен к структуре строки: на random-like он работает быстро, потому что расширение почти сразу останавливается, а на uniform время резко растёт.

Наиболее показательный результат получен для UNI_16000_A: Манакер выполнился за 1.419 мс, а расширение от центра — за 270.883 мс. Ускорение составило примерно 190.83 раза. Это подтверждает, что для строк с большим числом вложенных палиндромов линейный алгоритм принципиально предпочтительнее.

3.4 Сравнительная таблица применимости алгоритмов

По результатам анализа сформирована сводная таблица рекомендаций по применению алгоритмов в зависимости от типа и размера строки (табл.3.2).

Таблица 3.2

Сравнительная применимость алгоритма Манакера и расширения от центра

Сценарий	Манакер	Расширение от центра	Рекомендация
----------	---------	----------------------	--------------

Сценарий	Манакер	Расширение от центра	Рекомендация
Псевдослучайная строка до 10000 символов	Применим	Применим	Оба алгоритма дают близкое время; Манакер предпочтителен как универсальный
Зеркальная строка до 10000 символов	Применим	Применим	Манакер быстрее при росте длины
Чередующаяся строка до 5000 символов	Применим	Применим	Манакер быстрее на регулярной структуре
Строка из одинаковых символов до 16000 символов	Применим	Медленно	Использовать Манакер; эталон годится только для проверки
Строка существенно больше 16000 символов	Применим	Нецелесообразно	Использовать только Манакер
Функциональная проверка малых строк	Применим	Применим	Расширение от центра удобно как простой oracle для unit- и BDD-тестов

3.5 Выводы

Проведённый анализ позволяет сформулировать следующие выводы.

Во-первых, алгоритм Манакера стабильно быстрее расширения от центра на регулярных строках, где присутствует большое число вложенных палиндромов. На наборе UNI_16000_A ускорение составило примерно 190.83 раза.

Во-вторых, расширение от центра может быть конкурентоспособным на строках без длинных палиндромов, например на RND_10000_A. Однако этот результат не отменяет худшую квадратичную оценку алгоритма.

В-третьих, наблюдаемые результаты соответствуют теоретической сложности: алгоритм Манакера имеет оценку $O(n)$, а расширение от центра — $O(n^2)$ в худшем случае. Поэтому для производственного анализа строк следует использовать алгоритм Манакера, а расширение от центра оставить как эталонную проверку корректности.

В-четвёртых, BDD-сценарии оказались удобны для фиксации пользовательского поведения и нагрузочного сравнения, но не заменяют unit-тесты, которые точнее проверяют массивы радиусов и граничные условия.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы была достигнута поставленная цель — изучены и применены на практике принципы BDD-тестирования, проведён сравнительный анализ производительности алгоритмов поиска палиндромных подстрок, а также выполнено сопоставление BDD и модульных тестов. Все задачи, сформулированные во введении, были выполнены.

Соответствие теоретических временных оценок экспериментальным данным. Алгоритм Манакера имеет теоретическую сложность $O(n)$, так как каждый символ строки обрабатывается ограниченное число раз. Алгоритм расширения от центра имеет сложность $O(n^2)$ в худшем случае, поскольку для каждого центра может выполняться длинное расширение. Экспериментальные замеры подтвердили эту оценку: на наборе UNI_16000_A, состоящем из одинаковых символов, Манакер выполнился за 1.419 мс, а расширение от центра — за 270.883 мс. Разница составила примерно 190.83 раза. На псевдослучайной строке RND_10000_A оба алгоритма показали близкое время, что также соответствует теории: расширение быстро останавливается при отсутствии длинных палиндромов.

Анализ сходства распределений времени выполнения. Сопоставление профилей времени выполнения двух алгоритмов на семи нагрузочных сценариях показало, что распределения качественно различаются. Алгоритм Манакера остаётся близким к линейному на всех типах строк, тогда как расширение от центра сильно зависит от структуры входа. На регулярных строках *alternate*, *mirror* и особенно *uniform* время эталонного алгоритма растёт значительно быстрее. Формальная проверка статистических гипотез не выполнялась, так как она относится к заданию повышенной сложности, однако наблюдаемая разница на худшем наборе является устойчивой и объясняется различием алгоритмических оценок.

Сравнение BDD и модульных тестов. В рамках работы для одного и того же приложения использовались как модульные тесты Jest, так и BDD-тесты Cucumber.js с языком Gherkin. Модульные тесты проверяют внутренние функции: массивы радиусов, восстановление палиндрома, валидацию JSON и форматирование результата. BDD-тесты проверяют поведение системы на уровне пользовательских сценариев: анализ строки, обмен данными через JSON и нагрузочное сравнение алгоритмов.

Сравнительные характеристики двух подходов сведены в табл.3.3.

Таблица 3.3

Сравнение BDD-тестов и модульных тестов

Критерий	Модульные тесты Jest	BDD-тесты Cucumber.js
Число тестов	64 теста в 7 test suite	10 сценариев, 57 шагов
Время выполнения	Около 2.5 с при запуске npm test	Около 0.05 с без учёта установки зависимостей
Читаемость	Требует знания JavaScript и Jest API	Понятны как исполняемая спецификация на Gherkin
Локализация ошибок	Точная, до функции или модуля	Сценарная, до нарушенного пользовательского поведения
Нагрузочное тестирование	Не является основной целью	Встроено в performance_comparison.feature
Переиспользование	Через helper-функции и test.each	Через общие Step Definitions

Итоговый вывод состоит в том, что BDD и модульные тесты не заменяют, а дополняют друг друга. Модульные тесты обеспечивают точную проверку алгоритмических деталей, а BDD-сценарии фиксируют пользовательскую ценность и служат читаемой исполняемой документацией. Для задачи поиска палиндромных подстрок наиболее эффективной оказалась комбинированная стратегия: алгоритм Манакера используется как основное производственное решение, расширение от центра — как эталон корректности, а BDD сценарии — как верхнеуровневая проверка поведения приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *North D.* Introducing BDD. — URL: <https://dannorth.net/introducing-bdd/> (visited on 27.05.2026).
2. Cucumber.js Documentation. — URL: <https://github.com/cucumber/cucumber-js> (visited on 27.05.2026).
3. Node.js Performance Measurement APIs. — URL: https://nodejs.org/api/perf_hooks.html (visited on 27.05.2026).
4. *Manacher G.* A New Linear-Time “On-Line” Algorithm for Finding the Smallest Initial Palindrome of a String // *Journal of the ACM*. — 1975. — Vol. 22, no. 3. — P. 346–351.
5. *Gusfield D.* Algorithms on Strings, Trees, and Sequences. — Cambridge University Press, 1997.
6. Gherkin Reference. — URL: <https://cucumber.io/docs/gherkin/reference/> (visited on 27.05.2026).
7. Cucumber Documentation. — URL: <https://cucumber.io/docs/> (visited on 27.05.2026).
8. The RSpec Book: Behaviour Driven Development with RSpec, Cucumber, and Friends / D. Chelimsky [et al.]. — Pragmatic Bookshelf, 2010.
9. *Гмурман В. Е.* Теория вероятностей и математическая статистика: учебное пособие для вузов. — 9-е изд. — Москва: Высшая школа, 2003. — 479 с.
10. Jest Documentation. — URL: <https://jestjs.io/docs/getting-started> (visited on 27.05.2026).
11. Manacher’s Algorithm — Finding all sub-palindromes in $O(n)$. — URL: <https://cp-algorithms.com/string/manacher.html> (visited on 27.05.2026).

Приложение 1**Листинги кода классов предварительного эксперимента**

scripts/stringDataset.js

scripts/performanceCore.js

scripts/generateDatasets.js

scripts/runPerformanceExperiment.js

Листинги BDD-сценариев и реализации шагов

features/palindrome_analysis.feature

features/json_io.feature

features/performance_comparison.feature

features/support/world.js

features/step_definitions/palindromeSteps.js